二、构建专属聊天机器人

这一章节，我们将使用langChain构建一个专属机器人。这个聊天机器人不光可以进行简单的聊天，还需要提供更多专属的聊天服务。

- 使用LangChain Expression Language(LCEL)实现多步骤的聊天互动
- 保存历史聊天记录，实现自动多轮聊天
- 聊天消息整合LCEL

1、LangChain Expression Language(LCEL)

一个强大的聊天机器人，当然不能是问一个问题就回一个答案的简单互动，需要能够整合各种大模型，构建复杂的聊天场景。这时，LangChain提供的LCEL表达式就派上了用场。
LangChain Expression Language(LCEL)是LangChain的表达式语言，它提供了一种简单的方式来定义和执行任务。例如，之前与AI大模型交互的过程，可以用LCEL来表示：

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from config.load_key import load_key
from langchain_openai import ChatOpenAI

# 提示词模板
prompt_template = ChatPromptTemplate.from_messages([
    ("system","Translat the following from English into {language}"),
    ("user","{text}")
])
# 构建阿里云百炼大模型客户端
llm = ChatOpenAI(
    model="qwen-plus",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key=load_key("BAILIAN_API_KEY"),
)

# 结果解析器 StrOutputParser会AIMessage转换成为str，实际上就是获取AIMessage的content属性。
parser = StrOutputParser()
# 构建链
chain = prompt_template | llm | parser
# 直接调用链
print(chain.invoke({"text":"nice to meet you", "language":"Chinese"}))
```

通过LangChain的LCEL链式语法，就可以直接构建更为复杂的基于大模型的处理链。例如将某一次大模型调用的结果再次访问另一个大模型，那就只需要再chain的后面再链接更多的组件即可。

```
# 继续构建更复杂的链
analysis_prompt = ChatPromptTemplate.from_template("我应该怎么回答这句话? {talk} 。给我一个五个字以内的示例")
chain2 = {"talk":chain} | analysis_prompt | llm | parser
print(chain2.invoke({"text":"nice to meet you", "language":"Chinese"}))
```

你甚至可以构建多个并行的链，构建更复杂的大模型应用逻辑。下面的案例就会将英文同时翻译成中文和法语，最后将多个翻译结果整合成一个完整的结果。

```
    Input
     / \
    /   \
Branch1 Branch2
    \   /
     \ /
    Combine
```

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableMap, RunnableLambda,
RunnableWithMessageHistory

# 提示词模板
prompt_template_zh = ChatPromptTemplate.from_messages([
    ("system", "Translate the following from English into Chinese"),
    ("user", "{text}")
])

prompt_template_fr = ChatPromptTemplate.from_messages([
    ("system", "Translate the following from English into French"),
    ("user", "{text}")
])

# 构建链
chain_zh = prompt_template_zh | llm | parser
chain_fr = prompt_template_fr | llm | parser

# 并行执行两个链
parallel_chains = RunnableMap({
    "zh_translation": chain_zh,
    "fr_translation": chain_fr
})

# 合并结果
final_chain = parallel_chains | RunnableLambda(lambda x: f"Chinese:
{x['zh_translation']}\nFrench: {x['fr_translation']}")

# 调用链
print(final_chain.invoke({"text": "nice to meet you"}))
```

接下来，结合LangGraph，也可以用图形化的方式查看链的运行情况。
首先需要安装langgraph库。

```
!pip install langgraph
!pip install grandalf
```

接下来就可以打印出图形化的链图。

```
final_chain.get_graph().print_ascii()
```

LangChain的链式语法功能非常强大。甚至可以根据用户输入的问题，动态构建不同的链。
另外，在LCEL中，除了可以链接示例中几个简单的组件之外，还可以链接更多组件，完成更复杂的应用逻辑。后面会逐渐接触到。

> 实际上，LangChain提供了一个顶级父类，Runnable。只要是Runnable的子类，就都可以链接成一个序列。

2、保存聊天记录

接下来，聊天机器人当然要保存聊天记录，这样不光可以让用户和机器人进行多轮对话，还可以根据聊天记录进行检索。
在LangChain中，提供了一个BaseChatMessageHistory父类，专门用于保存聊天记录。
默认情况下，LangChain只提供了一个基于内存的InMemoryChatMessageHistory来保存聊天记录。

```
from langchain_core.chat_history import InMemoryChatMessageHistory
# 这是BaseChatMessageHistory的子类
history = InMemoryChatMessageHistory()
# 第一轮聊天
history.add_user_message("你是谁？ ")
aimessage = llm.invoke(history.messages)
print(aimessage.content)
history.add_message(aimessage)
# 第二轮聊天 如果没有上一次聊天的记录，大模型是不知道要重复什么内容的。
history.add_message("请重复一次")
aimessage2 = llm.invoke(history.messages)
print(aimessage2.content)
history.add_message(aimessage2)
# 打印历史聊天记录
# print("Chat History:")
# for message in history.messages:
#     print(f"{type(message).__name__}: {message.content}")
```

当然，这种通过内存管理消息记录的方式是不太安全的，应用一重启，消息记录都丢了。所以，LangChain还提供了很多基于其他存储系统的扩展依赖。比如，可以基于Redis、MongoDB、PostgreSQL、MySQL等数据库来保存聊天记录。具体参见官网： https://python.langchain.com/docs/integrations/memory/
以Redis为例，将上面案例中的消息记录保存到Redis中，要做的，只是替换一下history的实现类。
首先需要安装langchain针对redis的扩展依赖。

```
!pip install -q langchain-redis redis
```

然后，替换history的实现。

> 另外，需要部署Redis服务，过程略过。

```
from langchain_redis import RedisChatMessageHistory
# 这也是BaseChatMessageHistory的子类。本地需启动Redis服务
history = RedisChatMessageHistory(session_id="test",redis_url="redis://localhost:6379/0")
# 第一轮聊天
history.add_user_message("你是谁？")
aimessage = llm.invoke(history.messages)
print(aimessage.content)
history.add_message(aimessage)
# 第二轮聊天
history.add_user_message("请重复一次")
aimessage2 = llm.invoke(history.messages)
print(aimessage2.content)
history.add_message(aimessage2)
```

运行完成后，就可以在Redis中查看到聊天记录。

这样，下次再启动应用，就可以直接从Redis中加载聊天记录，而不用重新对话。

> 补充：基于LangChain良好的设计，也可以通过继承的方式来扩展其他存储系统的依赖。例如用开发工具打开下BaseChatMessageHistory的源码，就能看到，在 BaseChatMessageHistory 源码的注释中就演示了，如何扩展出一个基于文件保存消息的实现类。

3、聊天消息整合LCEL

如果每次都要手动调用ChatMessageHistory组件保存消息，那肯定太麻烦了，所以通常可以通过LCEL来快速整合聊天记录。接下来，如何整合呢？由于 BaseChatMessageHistory 并没有实现Runnable类，所以他是无法直接接入到LCEL的。而在LangChain中，又另外提供了一个RunnableWithMessageHistory类，他实现了Runnable，专门用于整合聊天记录。

```
from langchain_core.runnables.history import RunnableWithMessageHistory

# 构建阿里云百炼大模型客户端
```

```
llm = ChatOpenAI(
    model="qwen-plus",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key=load_key("BAILIAN_API_KEY"),
)

runnable = RunnableWithMessageHistory(
    llm,
    get_session_history=lambda: history, # 匿名函数，返回history
    ) # 返回的key

#第一次聊天 清除历史聊天记录
#history.clear()

# runnable.invoke({"text":"你是谁"})
# 之后每次聊天时，会自动带上Redis中的聊天记录。
runnable.invoke({"text":"请重复一次"})
```

最后，当然也可以给MessageHistory整合一个Chain，来完成更复杂的应用逻辑。

```
from langchain_core.runnables.history import RunnableWithMessageHistory

from langchain_core.messages import SystemMessage

# 提示词模板
prompt_template = ChatPromptTemplate.from_messages([
    ("user", "{text}")
])
# 构建阿里云百炼大模型客户端
llm = ChatOpenAI(
    model="qwen-plus",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key=load_key("BAILIAN_API_KEY"),
)
# 结果解析器 StrOutputParser会AIMessage转换成为str，实际上就是获取AIMessage的content属性。
parser = StrOutputParser()
# 构建链
chain = prompt_template | llm | parser

runnable = RunnableWithMessageHistory(
    chain,
    get_session_history=lambda: history, # 匿名函数，返回history
    ) # 返回的key

# 第一次聊天，清除历史聊天记录
# history.clear()

# 每次聊天时，会自动带上Redis中的聊天记录。
# runnable.invoke({"text":"你是谁"})
runnable.invoke({"text":"请重复一次"})
```

总结

这一章节接触了langchain中两个重要的工具，LCEL和BaseChatMessageHistory。其中，BaseChatMessageHistory主要用来管理聊天记录，而这些聊天记录都是与AI大模型进行多轮长聊天的基础。LCEL则是LangChain中的一个重要串联器。通过LCEL可以把LangChain中各种组件灵活的组装到一起，这不光是简化了基于LangChain的应用编码，更是构建个性化智能体的基础。